
PART II — BUILDING THE INGESTION PIPELINE

Chapter 5B

Evaluating Document-Conversion Engines: Docling, Unstructured, and Marker-PDF

“The words can survive a conversion while the document's meaning does not.”

Chapter 5B — Evaluating Document-Conversion Engines: Docling, Unstructured, and Marker-PDF

Chapter 5 introduced loaders as the entry point from files into a RAG system. That treatment is sufficient when a loader returns faithful text in a dependable order. It is not sufficient for contracts, requests for tender, engineering drawings, forms, or book pages whose meaning depends on headings, merged cells, repeated page furniture, and visual grouping. For those documents, loading is an evaluation problem before it becomes an ingestion step.

This chapter reconstructs a real comparison of Docling 2.113.0, Unstructured 0.24.1, and Marker-PDF 1.10.2. The project used three small batch scripts, three isolated Python environments, mirrored result trees, and a 251-page Marker assessment rather than trusting package descriptions. By the end, you will be able to build a comparable conversion harness, interpret each engine's output model, detect silent structural failure, and decide whether generated Markdown is authoritative data or merely an evaluation artifact.

5B.1 Why revisit loading — when text is not enough

Definition — Conversion fidelity

The degree to which a converted representation preserves the source document's content, reading order, hierarchy, relationships, and required assets. Text fidelity is only one component of conversion fidelity.

A plain prose PDF can survive conversion even when styling disappears because its meaning is carried mostly by sentence order. A procurement form is different. If a converter places one organization's address in another organization's row, the words are present but the record is false. If an engineering title block becomes an ordinary paragraph, revision identity and approval status disappear. If a repeated page banner becomes an H1 heading on every page, a structure-aware splitter sees dozens of invented sections.

The original `UnstructuredLoader` path was therefore revisited for layout-dependent documents. The question was not, "Which library extracts the most words?" It was, "Which representation preserves enough meaning for the next operation?" The answer depends on downstream use. Search may tolerate flattened formatting; clause retrieval cannot tolerate corrupted hierarchy; compliance review cannot tolerate swapped table rows; and publication cannot tolerate missing figures.

Analogy — A photocopy of a transit map

A photocopy can preserve every station name while losing the colored lines and intersections that show how stations connect. A conversion engine can likewise retain nearly every word while destroying the relationships that make the document usable.

5B.2 The comparison harness

A fair comparison holds the source corpus and output organization constant while allowing each library to expose its native API. The harness recursively discovers every file under `source/`, converts it with one engine, and mirrors its relative path beneath that engine's result directory. Thus `source/legal/terms.pdf` becomes `docling_results/legal/terms.md`, `unstructured_results/legal/terms.md`, and `marker_results/legal/terms.md`. Equivalent outputs remain easy to locate without hiding library-specific behavior behind one abstraction.

```

PROJECT_ROOT = Path(__file__).resolve().parent.parent
SOURCE_DIR = PROJECT_ROOT / "source"
OUTPUT_DIR = PROJECT_ROOT / "loader_results"

def output_path_for(source_path: Path) -> Path:
    relative = source_path.relative_to(SOURCE_DIR)
    return (OUTPUT_DIR / relative).with_suffix(".md")

for source_path in sorted(SOURCE_DIR.rglob("*")):
    if not source_path.is_file():
        continue
    output_path = output_path_for(source_path)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    convert_one(source_path, output_path)

```

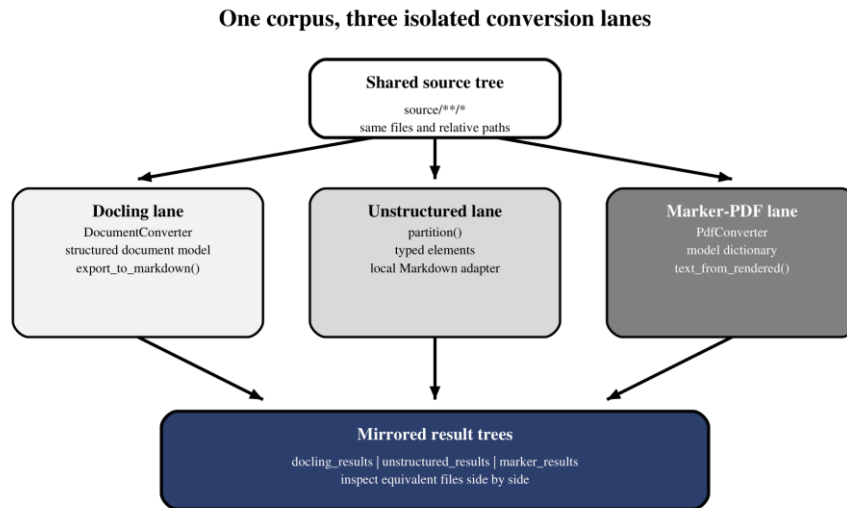


Figure 5B.1 — One shared corpus feeds three isolated but comparable conversion lanes.

Figure 5B.1 shows the essential experimental control: source identity is shared, while execution and outputs remain separate. Each batch continues after an individual file failure, reports converted and failed counts, and exits nonzero after partial failure. That design preserves successful evidence without letting a red summary line disappear.

A reproducible run also records the engine version, Python version, hardware path, source hash, output hash, start time, duration, and whether model assets were already cached. Cold-start time includes downloads and initialization; warm conversion time does not. Comparing one cold Docling run with one warm Marker run would measure setup state more than engine performance. Quality review should likewise use the same source pages and the same rubric for all engines.

The most useful review unit is not the whole result file but a set of deliberately difficult page types: one prose page, one simple table, one merged or cross-page table, one nested list, one form, one image-heavy page, and one page with repeated furniture. Reviewers record whether text survived, whether associations remained correct, and whether the representation is usable for the intended downstream task. This turns "looks better" into an inspectable decision.

Dimension	Why it must stay comparable	Harness control
Source selection	Different files invalidate quality comparisons	One recursive `source/**/*` tree
Path identity	Equivalent outputs must be findable	Mirrored relative paths
Failure handling	One bad file must not erase the batch	Per-file exception plus final nonzero exit
Output ownership	Engine behavior must remain visible	One result tree per loader
Inspection	File size alone cannot show structural quality	Side-by-side Markdown and source review

5B.3 Docling

Definition — Structured document model

An intermediate representation that stores recognized document elements and their relationships before serializing them to a format such as Markdown.

The Docling lane creates one `DocumentConverter`, converts each source, and calls `result.document.export\_to\_markdown()`. Its defining feature is the intermediate document model: layout analysis, OCR, table recognition, and reading-order decisions occur before Markdown export. The experiment explicitly selected the `PyPdfiumDocumentBackend` for PDF input while keeping the surrounding script small enough to read.

```
converter = DocumentConverter(
    format_options={
        InputFormat.PDF: PdfFormatOption(
            backend=PyPdfiumDocumentBackend
        )
    }
)

result = converter.convert(source_path)
markdown = result.document.export_to_markdown()
```

First execution may download or initialize layout and OCR assets, so cold-start time is not representative of steady conversion time. More importantly, a returned document is not proof of page completeness. In the observed run, native preprocessing reported `std::bad\_alloc` for individual pages while the batch still wrote Markdown and announced six converted, zero failed. The script's exception boundary saw a document-level success even though page-level work had failed.

Common pitfall — Treating batch success as page completeness

BUG-009 showed Docling returning a partial document after page preprocessing failures. Validate expected pages or page-level artifacts; do not equate a returned object with a complete conversion.

5B.4 Unstructured

The Unstructured lane calls `partition(filename=...)`, receiving typed elements rather than final Markdown. A deliberately lightweight adapter maps `Title` to H1, `Header` to H2, `ListItem` to a Markdown bullet, and every other element to a plain paragraph. This makes the serialization policy visible: any metadata or layout feature not handled by the adapter is discarded even if Unstructured detected it.

```
def elements_to_markdown(elements) -> str:
    lines = []
    for element in elements:
        text = str(element)
        if element.category == "Title":
            lines.append(f"# {text}")
        elif element.category == "Header":
            lines.append(f"## {text}")
        elif element.category == "ListItem":
            lines.append(f"- {text}")
        else:
            lines.append(text)
    return "\n\n".join(lines)
```

This distinction prevents an unfair conclusion. A weak Markdown file may reflect the local adapter rather than the partitioner's full capability. The experiment evaluates the delivered end-to-end path, not an abstract library maximum. It also records that the expanded six-PDF corpus was not run through Unstructured; only the retained RAG-chapter output supports direct comparison. That limitation belongs in the conclusion rather than being hidden.

Analogy — A customs declaration

The partitioner may inspect a suitcase in detail, but the adapter is the declaration form. If the form has only four categories, everything else is collapsed into "other" even though the inspector saw more.

5B.5 Marker-PDF

The Marker lane is PDF-oriented. It constructs a reusable `PdfConverter` with `create_model_dict()`, invokes the converter for each source, and extracts Markdown through `text_from_rendered()`. Unsupported file formats are reported as per-file failures. Model initialization is comparatively heavy, which is why the converter is created once per batch rather than once per document.

```
converter = PdfConverter(artifact_dict=create_model_dict())

rendered = converter(str(source_path))
markdown, metadata, images = text_from_rendered(rendered)
output_path.write_text(markdown, encoding="utf-8")
```

The final line in the real script writes only the Markdown string. The returned image collection is ignored. That small integration choice becomes a major quality finding later: the Markdown can reference detected

images without delivering their files. Marker also exposes model, PDF text, and rendering behavior through a large dependency stack, so a compact application script does not imply a simple runtime.

Engine	Native output used	Strength of this lane	Evaluation caution
Docling	Structured document model	Rich layout/OCR pipeline before export	Page-stage failure can hide below batch success
Unstructured	Typed partition elements	Transparent element categories	Local adapter discards unhandled structure
Marker-PDF	Rendered document to Markdown	Strong prose recovery and PDF focus	Assets and structure require separate validation

5B.6 Dependency isolation

The three engines could not share one reliable environment at the selected versions. Unstructured's `pi-heif` dependency required Pillow 11.1 or newer, while Marker-PDF 1.10.2 required Pillow below 11. An unpinned combined installation also selected a Numba release incompatible with Python 3.12, accumulated incomplete Torch metadata, and briefly installed the unrelated distribution named `marker`, which shadowed Datalab's `marker-pdf` namespace.

```
uv venv .venv-docling --python 3.12
uv pip install --python .venv-docling\Scripts\python.exe "docling==2.113.0"

uv venv .venv-unstructured --python 3.12
uv pip install --python .venv-unstructured\Scripts\python.exe
"unstructured[pdf]==0.24.1"

uv venv .venv-marker --python 3.12
uv pip install --python .venv-marker\Scripts\python.exe "marker-
pdf==1.10.2"
```

Isolation adds three setup commands but removes ambiguity. Each result is tied to a known package set, rebuilding one lane cannot damage another, and native ML packages no longer negotiate one impossible resolver solution. Python 3.12 was standardized because the newer Python 3.14 environment lacked compatible Windows wheels for parts of the OCR stack. PowerShell became the verified shell after Git Bash produced misleading native-process failures.

Common pitfall — Installing `marker` instead of `marker-pdf`

BUG-008 produced an import from an unrelated package with the same top-level namespace. Install `marker-pdf` in its own environment and verify the expected `PdfConverter` import.

Analogy — Three laboratories sharing one reagent cabinet

If three experiments require mutually incompatible chemical grades, one shared cabinet creates contamination and mislabeled results. Separate environments are sealed benches: slightly more setup, far stronger provenance.

5B.7 Reading the Marker-PDF quality report

The main assessment compared six PDFs totaling 251 pages: procurement forms, legal agreements, engineering scopes and specifications, and a conventionally designed RAG chapter. It separated content extraction from structural and visual reconstruction. Marker recovered ordinary prose impressively. The 91-page legal document yielded about 44,443 Markdown words from roughly 45,450 extractable PDF words; the RAG chapter yielded about 4,032 from roughly 4,023. Bold, italics, footnotes, colored text, and many simple tables survived.

The failure pattern appeared wherever two-dimensional relationships carried meaning. Merged headers, changing column counts, narrow cells, borderless groups, forms, engineering title blocks, and cross-page tables flattened badly. Heading levels were inconsistent: running banners, definitions, subordinate labels, and ordinary sentences became H1-H4. Repeated headers and footers entered the reading stream, page breaks disappeared, and nested list depth weakened. On engineering documents, dozens of genuine diagrams and page images were represented only by broken links.

Source pattern	Observed result	Downstream consequence
Linear prose	Most words and emphasis retained	Useful for search after cleanup
Simple fixed-column table	Often usable as a pipe table	Still validate row associations
Merged or cross-page table	Rows, columns, or sequence corrupted	Unsafe for clause/compliance extraction
Form or signature layout	Spacing and grouping flattened	Fields lose operational meaning
Engineering page frame	Title blocks and graphics lost	Revision and visual evidence unavailable
Repeated page furniture	Mixed into headings and lists	False boundaries and noisy chunks

The key lesson is conditional adoption. Marker is a strong content-oriented extractor for conventional prose, not a faithful page reconstruction engine. One variable-column table succeeded while another failed, so a simplistic rule such as "reject every complex-looking table" is not enough. Quality gates must inspect the delivered structure, especially table and page boundaries, rather than infer reliability from file type alone.

Worked example — the contact table

On page 5 of the first long RFT volume, the source contact table associated an organization, department, and contact details within one row. Marker expanded one principal row into several Markdown rows, split 'Contracts Department' across rows, and merged 'ADNOC LNG' into the preceding row. A bag-of-words comparison would score the page highly because almost every token remained. A row-association check would fail it immediately because the output now asserts relationships the source did not.

For retrieval, that distinction is decisive. A query for the organization's contact can retrieve a chunk containing all expected words yet return the wrong pairing. The acceptance test should therefore sample table keys and verify that each key remains attached to its value, check consistent column counts, and flag suspicious row expansion. Where merged cells or spatial grouping carry meaning, the system should preserve HTML or structured cell coordinates rather than force the page into a pipe table.

5B.8 The two silent-failure modes

Definition — Silent failure

A conversion defect that does not surface as a failed process or exception, allowing incomplete or corrupted output to be treated as successful.

The first silent failure is high word retention masking structural corruption. In the long RFT continuation, a dense table around pages 61-62 was reordered, section `2.1.2` appeared twice, and later content began disappearing. Another contact table preserved most words while merging organizations into the wrong rows. Aggregate counts looked healthy because they measure lexical volume, not correct associations, sequence, or completeness.

The second is packaging failure. The six-file Marker run produced 131 image references and zero corresponding image assets. The RAG chapter also contained at least 26 mojibake sequences despite UTF-8 file writes. The core model may have detected image regions and text correctly, but the evaluated end-to-end workflow delivered broken Markdown. Users consume the result tree, not the converter's internal intention.

Both modes are detectable with inexpensive automation. Compare expected page identifiers or known anchors against output; parse every Markdown image destination and confirm that it resolves beneath an approved asset directory; scan for replacement characters and common mojibake byte patterns; count headings per page and table widths per row; and retain a short list of source passages whose relative order must remain stable. These checks do not prove fidelity, but they turn several previously silent defects into explicit rejection reasons.

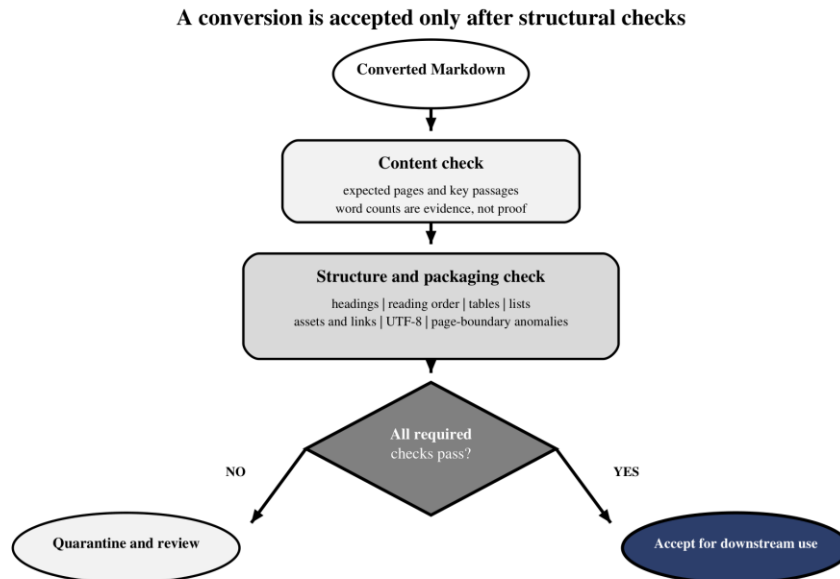


Figure 5B.2 — An acceptance gate checks content, structure, assets, and encoding before downstream use.

Figure 5B.2 turns that lesson into a release rule. Expected words and pages are useful signals, but acceptance also requires heading, list, table, reading-order, link, asset, and encoding checks. A failed check should quarantine the document for targeted review rather than allow clean-looking Markdown to enter retrieval silently.

Common pitfall — Using file size or word count as a quality score

BUG-011 retained most words while corrupting row relationships and sequence. Count metrics can detect gross loss, but only source-to-output structural checks can validate meaning.

5B.9 The adoption decision

Definition — Authoritative representation

The version of a document that downstream systems are permitted to treat as the reliable source for meaning, relationships, and decisions.

The project kept raw converter Markdown as an evaluation utility rather than declaring it authoritative. This is not a rejection of the engines. It is recognition that the current runners omit acceptance machinery. A production route would preserve page and block metadata, export assets to stable relative paths, validate every link, normalize encoding, remove or tag page furniture, repair heading hierarchy, test tables for implausible geometry, and send structurally complex pages to manual review or quarantine.

- Validate source and output page coverage, not only process exit status.
- Preserve page, section, bounding-box, and source-path metadata beside Markdown.
- Export every referenced asset and fail acceptance on unresolved links.
- Scan for mojibake, implausible heading counts, one-letter cells, and inconsistent table widths.
- Compare high-risk forms and cross-page tables against source pages.
- Route low-confidence documents to a richer representation or human review.

The comparison harness remains valuable precisely because it does not conceal these differences. It can be extended into routed ingestion: inspect format and layout, choose an appropriate parser, normalize into `Document` objects, and quarantine uncertain results. That direction was researched but not implemented, so the chapter distinguishes recommendation from current architecture.

```
def assess_conversion(source, markdown, result_root):
    findings = []
    findings += check_expected_pages(source, markdown)
    findings += check_heading_distribution(markdown)
    findings += check_table_shapes(markdown)
    findings += check_asset_links(markdown, result_root)
    findings += check_encoding(markdown)

    if any(item.severity == "high" for item in findings):
        return {"status": "quarantine", "findings": findings}
    return {"status": "accepted", "findings": findings}
```

The skeleton deliberately returns findings rather than a single opaque score. A document can be excellent for full-text search and unacceptable for form reconstruction at the same time. The caller should decide acceptance against a declared use profile, while the findings preserve why the decision was made.